

A Formalisation of ASN.1

Eötvös Loránd University (ELTE)

Christian Rinderknecht

4 May 2014

Abstract Syntax Notation One

The wide variety of hardware architectures and programming languages calls for the definition of a universal format for data to be exchanged over a computer network.

Abstract Syntax Notation One (ASN.1) is a standardised language (ISO, ITU-T) for defining data types whose values may be exchanged across a network between two, possibly heterogeneous peers.

It is an ISO and ITU-T standard.

ASN.1 is completed by a set of *encodings* which serialise the ASN.1 values into streams of bits, the two most prominent being

- ▶ the Basic Encoding Rules (BER),
- ▶ and Packed Encoding Rules (PER).

Abstract Syntax Notation One (cont.)

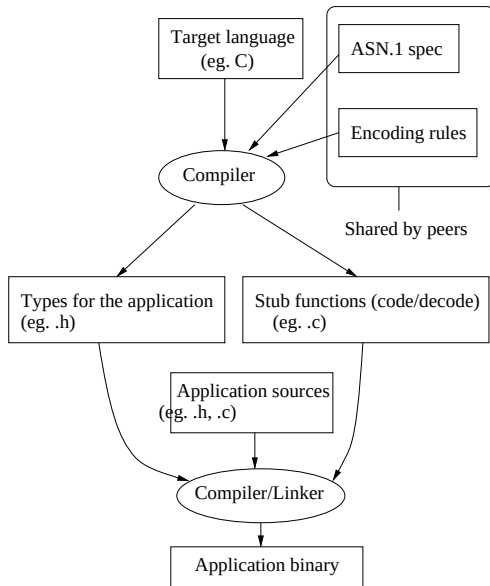
ASN.1 is used in network management, secure email, mobile telephony, wireless applications, Long Term Evolution (4G LTE), electronic commerce, air traffic control, voice and video over the internet, digital certificates, radio paging, interactive television, financial service systems, biometrics, ATM transactions, call routing to local carriers (USA), plane take-offs and landings, Federal Express tracking network, Microsoft Internet Explorer, Outlook, databases (genetics, libraries), diagnostic monitoring systems (car factories), metro equipments etc.

Abstract Syntax Notation One (cont.)

ASN.1 is embedded in other specification and programming languages like System Design Language (SDL, control), Message Sequence Charts (MSC, functional requirements), Testing and Test Control Notation (TTCN-3, tests suites), etc.

ASN.1 is also used without formal control specification by protocol designers and tool implementors, e.g., SNMP, OpenSSL.

ASN.1 and the compilation chain



How do ASN.1 and the BER interact?

- ▶ Both peers share a common ASN.1 specification and agree upon the usage of the BER;
- ▶ each peer compile the ASN.1 specification to a set of type declarations and a set of coders/decoders (codecs) for each type;
- ▶ the target language may be different depending on the peer;
- ▶ the coders transform a value into a stream of bits according to the BER,
- ▶ the decoders transform a stream of bits into a value according to the BER (or reject it as ill-formed);
- ▶ these pieces of source code are compiled and linked separately against the corresponding communicating applications.

Short presentation of ASN.1

Primitive types

- ▶ `ok BOOLEAN ::= TRUE`
- ▶ The NULL type has only one value, also noted NULL;
- ▶ `zero INTEGER ::= 0`
`DayInTheYear ::= INTEGER {first(1), last(356)}`
`newYearsEve DayInTheYear ::= last`
- ▶ `SynchroIndicator ::= ENUMERATED {serial,parallel}`
`synchro SynchroIndicator ::= serial`
- ▶ `pi REAL ::= 3.14159`
`micron REAL ::= {mantissa 1,base 10,exponent -6}`

More primitive types

- ▶ `byte BIT STRING ::= '0D'H -- or '00001110'B`
`T ::= BIT STRING {msb(7), lsb(0)}`
`v T ::= {msb, lsb} -- or '10000001'B`
- ▶ The OCTET STRING type is similar to the BIT STRING for eight-bit multiples strings.
- ▶ The OBJECT IDENTIFIER and RELATIVE-OID types aim at referencing other ASN.1 modules (a set of type and value declarations) at an international level, by means of a path in a standard tree.
- ▶ For historical reasons, there are many string types.

Some constructed types

The SET type corresponds to the record-like structures in programming languages:

```
PersonInfo ::= SET {age INTEGER, married BOOLEAN}  
i PersonInfo ::= {married TRUE, age 42}
```

Some fields can be marked as optional or having a default value:

```
Point ::= SET {x REAL DEFAULT 0, y REAL DEFAULT 0}  
origin Point ::= {} -- or {x 0.0, y 0.0}
```

A real example:

```
DataAcknowledgementTPDU ::= SET {  
    destRef          Reference,  
    yr-tu-nr         TPDUnumber,  
    checkSum         CheckSum OPTIONAL,  
    subSeqNr         SubSequenceNumber DEFAULT 0,  
    flowControlCnf   FlowControlConfirmation OPTIONAL}
```

Some constructed types (cont.)

The SET OF type corresponds to the mathematical notion of sets with repetition:

```
T ::= SET OF INTEGER
```

```
empty T ::= {}
```

```
small T ::= {7, 9, 1, 1, 3}
```

Some constructed types (cont.)

The CHOICE type corresponds to a **union** in C.

```
T ::= CHOICE {x REAL, y BOOLEAN}
```

```
u T ::= x : 0.5
```

```
v T ::= y : FALSE
```

The Protocol Data Units (PDU) are CHOICE types, because they model all the possible queries and responses between two peers. A CHOICE type may be recursive, like the other constructed types. One real example (a Network Management Protocol) is:

```
CMISFilter ::= CHOICE {  
    item  FilterItem,  
    and   SET OF CMISFilter,  
    or    SET OF CMISFilter,  
    not   CMISFilter}
```

Some subtyping constraints

ASN.1 offers a very involved subtyping paradigm consisting of constraints upon recursive types, that restricts their corresponding sets of values in a set-theoretic manner, but also in a structural way.

- ▶ **Interval Constraint:** the INTEGER, REAL and (almost all) string types have totally ordered values, hence allowing interval definitions. For instance:

```
PositiveOrZeroInteger ::= INTEGER (0..MAX)
```

```
PositiveInteger ::= INTEGER (0<..MAX)
```

```
NegativeOrZeroInteger ::= INTEGER (MIN..0)
```

```
NegativeInteger ::= INTEGER (MIN.. $<0$ )
```

```
PositiveReal ::= REAL (0<..PLUS-INFINITY)
```

```
NegativeReal ::= REAL (MINUS-INFINITY.. $<0$ )
```

```
RealInterval ::= REAL (4e-5..1e-4)
```

Some subtyping constraints (cont.)

- ▶ **Value Constraint:** to restrict the set of values of a type to be a singleton:

```
Wednesday ::= Day (wednesday)
```

- ▶ **Union Constraint:** the new subtype contains the values of the first subtype and of the second subtype (keyword UNION or symbol |):

```
Day ::= ENUMERATED {monday, tuesday, wednesday,  
                    thursday, friday, saturday, sunday}
```

```
WeekEnd ::= Day (saturday | sunday)
```

- ▶ **Alphabet Constraint:** the strings can be restricted to be built upon a given alphabet:

```
CapitalAndSmall ::=
```

```
  IA5String (FROM ("A".."Z" | "a".."z"))
```

```
CapitalOrSmall ::=
```

```
  IA5String (FROM ("A".."Z") | FROM ("a".."z"))
```

Some subtyping constraints (cont.)

- **Size Constraint:** the values of string types may be constrained to a given sizes, introducing a constraint by the keyword `SIZE`:

```
Exactly31BitsString ::= BIT STRING (SIZE (31))
```

```
StringOf31BitsAtTheMost ::=  
    BIT STRING (SIZE (0..31))
```

```
NonEmptyString ::= BIT STRING (SIZE (1..MAX))
```

The size constraint can also apply to `SET OF` types. In that case, the semantics is very different: the values of the types are sets whose *cardinals* are specified by the size constraint:

```
SetOf5Strings ::=
```

```
    SET (SIZE(5)) OF PrintableString
```

```
SetOfStringsOf5Char ::=
```

```
    SET OF PrintableString (SIZE(5))
```

Some subtyping constraints (cont.)

- ▶ **Intersection Constraint:** the new subtype contains the values that belong to the two subtypes (keyword `INTERSECTION` or symbol `^`):

```
FrenchPhoneNumber ::=
  NumericString (FROM ("0".."9") ^ SIZE (10))
```

- ▶ **Inclusion Constraint:** to restrict a subtype to have only the values of a given subtype (optional keyword `INCLUDES`):

```
LongWeekEnd ::=
  Day ((INCLUDES (WeekEnd)) | monday)
Bis ::= Day (WeekEnd | monday)
```

Some subtyping constraints (cont.)

- ▶ **Complement Constraint:** to restrict the values of a type to *not* belong to another subtype:

```
Lipogramme ::=
  IA5String (FROM (ALL EXCEPT ("e" | "E")))
```

- ▶ **Constraint on SET OF:** to restrict the elements of a SET OF value:

```
TextBlock ::= SET OF VisibleString
AddressBlock ::=
  TextBlock (WITH COMPONENT (SIZE(1..32)))
```


Some subtyping constraints (cont.)

- ▶ Partial Constraint: to restrict *some* fields of a SET or CHOICE:

```
Quadruple ::= SET {  
    alpha ENUMERATED {in, out} OPTIONAL,  
    beta  IA5String OPTIONAL,  
    gamma SET OF INTEGER,  
    delta BOOLEAN DEFAULT TRUE}
```

we can derive a subtype whose component alpha is always present and equals in, and the component gamma always has five elements:

```
Quadruple1 ::=  
    Quadruple (WITH COMPONENTS {...,  
                                                alpha (in) PRESENT,  
                                                gamma (SIZE (5)))})
```

Some subtyping constraints (cont.)

This subtype has the same values as:

```
Quadruple1 ::= SET {  
  alpha ENUMERATED {in, out} (in),  
  beta  IA5String OPTIONAL,  
  gamma SET SIZE (5) OF INTEGER,  
  delta BOOLEAN DEFAULT TRUE}
```

What are the Basic Encoding Rules (BER)?

- ▶ The BER are a set of rules defining the serialization into bits of ASN.1 values;
- ▶ the bit frames (*codes* or *encodings*) have a uniform structure: a triple made of a *tag*, a *length* and some *contents*;
- ▶ the tag field is extracted from the data type,
- ▶ the contents field is extracted from the data itself,
- ▶ the length field is computed over the contents field,
- ▶ the structure allows embedding of sub-frames (in the contents field)
- ▶ and an *indefinite length* form, as a sender's option.

Issues

- ▶ Specifications: huge (704 pages A4 10pt), complex (concepts mutually dependent).
- ▶ Syntax: large and ambiguous grammar, dynamic extensions.
- ▶ English cannot tell the meaning of all combinations of syntactical constructs.
- ▶ Types are value sets: subtypes yield general set constraints.
- ▶ Encoding only applies to a subset of ASN.1.
- ▶ Decoding is unspecified.
- ▶ Dynamic checks in codecs (depending on the target programming language) are unspecified.
- ▶ Open source or free compilers: extremely rare, unmaintained, fragile, without precise error reporting.

Issues in validation

ASN.1 types are considered as sets of values, thus types must have at least one (finite) value.

Because of the great expressivity of ASN.1, the compilers are not likely to fully check arbitrary combinations of subtyping constraints!

- ▶ Vendors' argument: "This is hardly a real problem because those critical specifications are rarely found in practice."
- ▶ My argument: "Maintenance costs are always wildly underestimated. Why not solve completely the problem, get a better product and save money and time?"

Practical Problems (with compilers)

- ▶ An incorrect specification is badly rejected:

`T ::= T` produces `typedef T T;`
before being rejected by the C compiler!

- ▶ A correction specification is rejected:

`T ::= SET (WITH COMPONENT (0)
 | WITH COMPONENT (1)) OF INTEGER`

- ▶ An incorrect specification is accepted:

`a INTEGER (0<..9) ::= 0`

So, what is wrong?

- ▶ In the last few years, the press has reported several alleged vulnerabilities
- ▶ relating ASN.1 and the BER to SNMP and OpenSSL
- ▶ about improper decoding (and rejection) of ill-formed BER encodings
- ▶ leading to buffer overflows,
- ▶ unspecified (non-deterministic) behaviours,
- ▶ stack corruptions,
- ▶ hence possible denials of service.
- ▶ Further study proved that all the problems lay actually in *implementations*:
- ▶ *we want to show that ASN.1 and the BER are, themselves, well designed.*

Incompleteness of the standard

- ▶ Aliasing

a T1 ::= *<a value of type T1>*

b T2 ::= a

What conditions on T1 and T2?

- ▶ Recursive types and values:

T ::= SET OF T

x T ::= {x}

y T ::= {{}}

Theoretical Problems

1. the types may have only infinite values:

`T ::= SET {a T};`

2. some value declarations may be ill-typed:

`v REAL ::= "";`

3. especially, some value references may be ill-typed, as

`a VisibleString ::= b`

`b INTEGER ::= 0;`

4. the subtype constraints may be inconsistent:

`T ::= REAL (SIZE(7));`

5. the subtypes may be empty, as

`T ::= SET ((SIZE (1)) ^ (SIZE (2))) OF REAL;`

6. the subtypes may have no value set:

`T ::= REAL (ALL EXCEPT T).`

Problems (cont.)

In other words, the issues to settle are respectively:

1. the finiteness problem;
2. the typechecking problem;
3. the type compatibility problem;
4. the constraint consistence problem;
5. the non-emptiness problem;
6. the solvability problem.

Analysis

- ▶ type compatibility $\subseteq$ typechecking;
- ▶ constraint consistence, non-emptiness $\subseteq$ solvability (the system is solved when we construct explicitly the values of each subtype);
- ▶ typechecking $\subseteq$ solvability:

$$y \text{ INTEGER } (0..9) ::= 1 \rightarrow \begin{cases} y \ A ::= 1 \\ A ::= \text{INTEGER } (0..9) \\ B ::= A \ (y) \end{cases}$$

where A and B are fresh type references.

So, finiteness and solvability are enough to get a full validation of ASN.1.

Advantages of formalisation

1. For the standardisation committees:
 - ▶ Help find incompletenesses, ambiguities, inconsistencies and unneeded restrictions;
 - ▶ If an encoding is also formalised, proofs of desirable properties, like correctness, become possible.
2. For the specifiers:
 - ▶ The formalisation can also define the algorithms for the front-end of an ASN.1 compiler.

Modelling (assumptions and encoding)

We start making some assumptions that abstract away low-level details and help us capturing the design principles of the BER:

- ▶ values are given in ASN.1, not in the application language (in fact they are produced in memory at run-time);
- ▶ codes are tuples, not bit series, in order to easily reason by induction on their structure;
- ▶ the network is reliable (no alteration of codes).

BER codes may not be unique for a given value, due to possible sender's options (e.g., encoding of booleans or unordered sets).

Proposition

All the BER encodings of a given value, according to a given type, are equivalent.

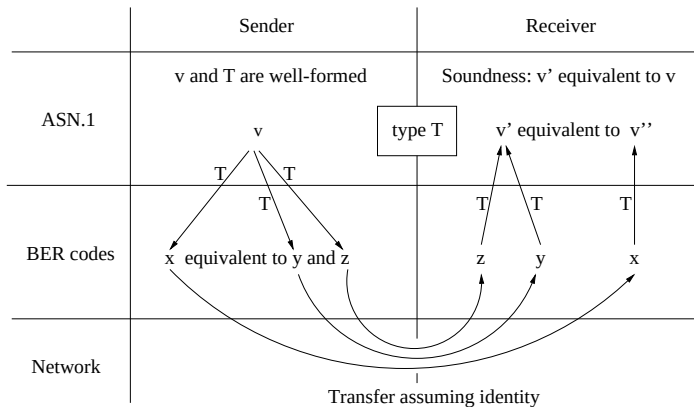
Modelling (decoding)

- ▶ The BER standard (26 pages of English) does not define the decoding at all.
- ▶ We believe that the receiver is the peer most concerned with the sound design of the BER: *the composition of encoding and decoding yields a value equivalent to the original one.* Hence we need an equivalence relationship between values (thus independent from the BER).

Theorem (Soundness)

Let v be a value of type T . Then the BER decoding of any BER encoding of v is equivalent to v .

Soundness property



Equivalence entailment

As a consequence, in order to prove the soundness property, it appears that we need to link logically the two equivalence relationships we defined separately on codes and values.

Proposition (Equivalence entailment)

Let c_1 and c_2 be two equivalent codes. Then the decoding of c_1 is equivalent to the decoding of c_2 assuming the same type.

One equivalence rely solely on the BER and the other one is inherent to ASN.1. Hence *this proposition is the link between the design of ASN.1 and the design of the BER.*

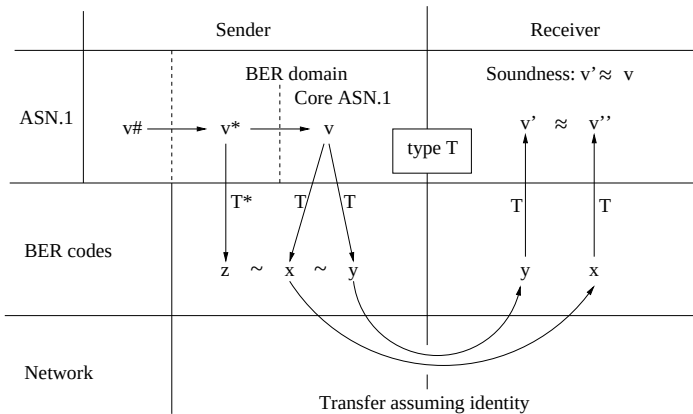
Core ASN.1

- ▶ The BER only apply to a subset ASN.1 (whose definition spans 188 pages).
- ▶ This suggests that the whole ASN.1 can be reduced to an inner subset which has the same expressivity, we call *BER domain*.
- ▶ In fact it is even possible to reduce further the BER domain to a smaller subset, we call *core ASN.1*, in which formal reasoning is easier.
- ▶ The formal definition of these reductions is achieved by means of a series of rewriting rules that preserve the set of values of the specifications (or of a given type).
- ▶ We can easily check some useful properties in core ASN.1:

Theorem (BER termination)

The encoding of core ASN.1 values with the BER always terminates.

Core ASN.1 and soundness property



Back to our model

- ▶ Let us note $\Gamma \vdash v : T \rightarrow c$ the judgement “In the environment Γ , the value v is encoded into the code c , following the type T .” (The environment models the ASN.1 module.)
- ▶ Let us note $\mathcal{D}(\Gamma, c, T)$ the decoding of c in the environment Γ according to type T .
- ▶ We already noted $v_1 \approx v_2$ the equivalence of values v_1 and v_2
- ▶ and $c_1 \sim c_2$ the equivalence of codes c_1 and c_2 .

The next step consists in defining these relations by means of systems of inference rules.

Formal model

We can now rephrase formally our propositions:

- ▶ Proposition “All the BER encodings of a given value, according to a given type, are equivalent.” becomes

Proposition

If $\Gamma \vdash v : T \rightarrow c_1$ and $\Gamma \vdash v : T \rightarrow c_2$ then $c_1 \sim c_2$.

- ▶ Proposition “The decoding of two equivalent codes lead to two equivalent values.” becomes

Proposition (Equivalence entailment)

$c_1 \sim c_2 \implies \mathcal{D}(\Gamma, c_1, T) \approx \mathcal{D}(\Gamma, c_2, T)$

- ▶ Theorem “Let v be a value of type T . Then the BER decoding of *any* BER encoding of v is equivalent to v .” becomes

Theorem (Soundness)

If $\Gamma \vdash v : T \rightarrow c$ then $\mathcal{D}(\Gamma, c, T) \approx v$.

Two approaches for the validation

Every ASN.1 specification is rewritten in a subset of same expressiveness, with less syntactical constructs. Two strategies:

1. Few preliminary rewrites, followed by a collection of set constraints, then fed to a generic constraint solver.
2. Lots of preliminary rewrites, simple algorithms to check solvability. (This is like taking the previous method and interleave a dedicated solver with the syntactical rewrites.)

Comparison:

1. Pros: modularity, less programming. Cons: constraints too general for off-the-shelf solvers.
2. Pros: easy checking. Cons: heavily specialised, correctness hard to assess.

We need to find a trade-off.

Set Constraints

Sets constraints are inclusions between expressions interpreted over the domain of sets of trees which may be recursively defined.

The ASN.1 types, by having a tree structure (technically, they are rational trees), fit perfectly in the usual domain of set constraints.

A set constraint has the form $\alpha \subseteq \beta$, where α and β are *set expressions*. Examples are 0 (the empty set), 1 (the set of all terms), α (a set-valued variable), $c(\alpha, \beta)$ (a constructor application), and the union, intersection or complement of set expressions. Given a set of constructors $\mathcal{C}$, where each $c \in \mathcal{C}$ has arity $a(c)$, set expressions are defined by the grammar:

$$E ::= 0 \mid 1 \mid \alpha \mid c(E_1, \dots, E_{a(c)}) \mid E_1 \cup E_2 \mid E_1 \cap E_2 \mid \neg E_1.$$

Common Application

Set constraints are used in program analysis: sets of terms describe the possibly computed values. Set constraints are generated from the program text and solving the former yields some useful information about the latter (e.g., type-checking or optimisation).

For example, consider a program with two uses of a variable v . Assume that from one use we determine that v must be a number (i.e., $v \subseteq \text{Int} \cup \text{Float}$) and from the other use that v cannot be an integer (i.e., $v \subseteq \neg \text{Int}$). Combining these constraints, we can infer that v must be a floating point number (i.e., $v \subseteq (\text{Int} \cup \text{Float}) \cap \neg \text{Int} = \text{Float}$).

Application to ASN.1

In order to apply the set constraints to ASN.1 we only define specific constructors c that model all the subtyping constraints:

Now the problem is to extract set constraints from each subtype, such as their solving provide a mapping from the subtype to its set of values: this is a *denotational semantics* of ASN.1.

As far as validation is concerned, if some constraints are unsolvable or the set of solutions is empty, then the specification must be rejected.

Aiken and Wimmers proposed the first solving algorithm for the class of constraints we are using.